



- Introduzione
- Machine Learning
 - Classificazione Supervisionata
 - Lezione extra
 - Supervised learning
 - Regressione
 - Regressione lineare
 - Regressione lineare con più variabili
 - Regressione polinomiale
 - Coefficienti di Pearson
 - Training set, test
 - Logistic Regression
 - Cost Function
 - Decision Boundary
 - Decision Boundary non lineare
 - Confusion Matrix - Matrice di confusione
 - Deduzione della performance del modello da una matrice di confusione
 - Confusion Matrix con più classi
 - Text Data
 - PreProcessamento
 - Bag of Word Approach
 - TF - IDF (Term Frequency - Inverse Document Frequency)
 - Word Representations
 - Word Embeddings
 - Holdout method and Regularization
 - K-Fold Cross Validation
 - Regularization
 - Regressione lineare 2D e classificazione
 - Regularization: Ridge regression
 - Regularization: Logistic regression
 - Bias-Variance
 - Learning curve
- Neural Networks
 - Modifica della logistic regression
 - Multilayer Neural Networks
 - Multilayer Neural Networks: Training



Introduzione

Gli appunti e lezioni si trovano solo su Teams

Orario ricevimento: mercoledì 11-12

Argomenti principali:

- Robe fatte col Maddalena + introduzione alle reti neurali e al deep learning

Esame:

- Uno scritto e un orale
- Per accedere all'orale bisogna superare lo scritto (voto minimo 18/30)
- aver passato lo scritto consente di accedere all'orale entro un anno accademico (fino a febbraio 2027 in questo caso)
- test in laboratorio (su elearnig)
 - esercizi dove va inserita la risposta
 - opzionale consegnare un foglio con il ragionamento (non è obbligatorio, ma è consigliato)
 - consentita solo la calcolatrice (non saranno comunque calcoli complessi)
- chi non passa l'esame orale deve rifare tutto
 - all'orale si cerca di tarare meglio il voto

Tutti gli appunti che saranno circondati da ()★ sono mie aggiunte ritenute interessanti o considerazioni personali. **NON** fanno parte del corso e (spero)★ non verranno chiesti all'esame. Sono comunque interessanti, quindi dateci un'occhiata.



Machine Learning

Fino a diversi anni fa la scienza non aveva intuito come creare macchine simili a chat gpt o gemini, anche se vi sono stati diversi tentativi. L'idea iniziale prevalente era quello di rappresentare la conoscenza con dei formalismi e poi risolvere dei problemi.

Per esempio, per riconoscere un volto, si cercava di scomporlo in elementi più semplici (naso, occhi, bocca) e successivamente si cerca di riconoscere questi elementi, con dei vincoli scorrevoli (detti molle).

Artificial intelligence -> machine learning -> deep learning

Negli ultimi anni (dopo chat GPT) si sono raggiunti risultati impensabili, dove un sistema basato su reti neurali poteva raggiungere risultati impressionanti.

Ora ci si basa sull'esperienza, qualora essa porti a risultati migliori si dice che la macchina sta imparando.

Sotto categorie di machine learning:

- Supervised learning
- Unsupervised learning
- Reinforcement learning

Noi vedremo principalmente sui supervised learning.



Classificazione Supervisionata

Un sistema cerca di classificare e categorizzare gli oggetti in base alle proprie caratteristiche.

Se vogliamo per esempio classificare se qualcosa è o non è un elefante, gli fornisco una serie di immagini di elefanti e non elefanti, con le rispettive etichette (1 per elefante, 0 per non elefante). In questo modo la macchina impara a riconoscere le caratteristiche che distinguono un elefante da un non elefante.

Un esempio più complesso è quello di classificare un testo come positivo o negativo, soprattutto perché devo ottenere una serie di dati con le rispettive etichette.

Dopo un certo numero di esempi, la macchina traccia una funzione (di primo o più gradi) in grado di determinare se un nuovo elemento sarà positivo o negativo

La parte più complessa è quella di rappresentare i dati reali in una semplificazione come un piano cartesiano.

Se per esempio abbiamo 3 variabili, non si può più rappresentare i dati in un piano cartesiano, ma si può rappresentare in uno spazio tridimensionale. E non si avrà una retta, ma un piano che divide lo spazio in due parti.

Con ulteriori variabili la rappresentazione diventa sempre più complessa (detto spazio ad alta dimensionalità), ma il concetto rimane lo stesso

Se volessimo categorizzare un volto, nel caso più semplice, abbiamo un'immagine in bianco e nero (un valore che va da 0 a 255), con una risoluzione relativamente bassa (Ad esempio 600X800 pixel). Ci sono quindi 480000 pixel, e quindi 480000 variabili.



Lezione extra

A model if is able to understand something, is also capable of understanding it not encoded, not as we see (let's say an image). is quite different from us, who are neaded to see the image encoded to fully understand it.

They are also present the **multilayer perectorons (MLPs)** who are capable of rappresenting more complex functions, like a 3d space. we have to train em to a very variate samples of data, in a way to cover all the possible cases.

There are present the CNNs and PointNets, who are expert at classifing shapes, and they often the output is a class of appartence.

There are also present Transformers, who have 3 different subspaces (or more) that triest to recombine the extracted features from each sub process, each said "patch". they were originally built for language models, who rely on a serires of data, non single elements (let's say a single image). they have less data neaded to be trained in comparason to CNNs, but they are more computationally expensive.

the model to understand images, it starts from applayng some guassian noise to the image, and after a determinated number of steps it is able to learn how to invert this operation, and therefore to generate an image starting from noise. this is the principle of the **DDPMs**. they can be applied not only to images, but also other data tipes. such thype of learning is capable of learning more complex items compared to other models, at the cost of being more computationally expensive.

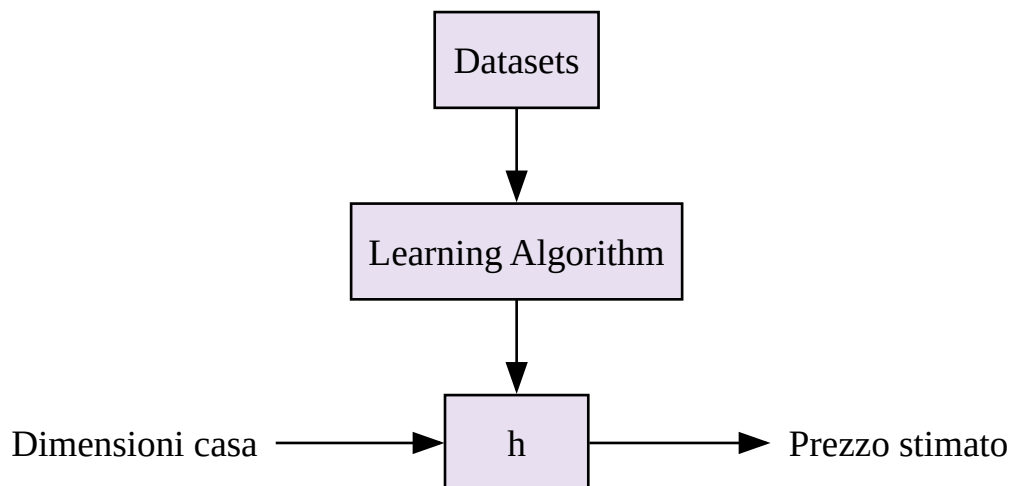


Supervised learning

Do in pasto alla macchina una serie di dati con le rispettive etichette, in modo che la macchina possa imparare a riconoscere le caratteristiche che distinguono un elemento da un altro.

Notazioni usate:

- m : numero di esempi di training
- $\mathbf{x}$: variabili di input (caratteristiche)
- $\mathbf{y}$: variabile di output (etichetta)
- (x, y) : coppia di input e output per addestrare la macchina
- $(x^{(i)}, y^{(i)})$: i -esima coppia di input e output





Regressione

Stimare un valore in base all'esperienza

Per le macchine l'esperienza si traduce in dati.

La regressione non deve essere necessariamente lineare, ma può essere anche di secondo o più gradi.

Bisogna stare molto attenti ad avere un sistema che non abbia dei "dati avvelenati", che magari rappresentano solo una parte del fenomeno, e quindi non riescono a generalizzare bene, o a discriminare.

Regressione lineare

formula più semplice:

$y = ax + b$, dove a è la pendenza e b è l'intercetta.

Per una forma più generale, con più variabili di input, si ha:

$$h_0(x_1) = \theta_0 + \theta_1 x_1$$

Viene più spesso rappresentata come nozione matriciale:

$$\text{Let's } \theta = [\theta_0 \ \theta_1] \quad x = [1 \ x_1]$$

E a seconda dei parametri θ che scegliamo, otteniamo una retta diversa.

(grafici)

Per calcolare la retta che meglio approssima i dati (si avvicina di più ai dati):

$$f(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Per addestrarlo, y_i è il valore presente all'interno del database, dove i è l'indice dell'esempio.

Una volta addestrato un modello, quando si inserirà un nuovo valore di x , il modello restituirà un valore di y stimato in base alla retta. Un buon modello è quello che riesce a stimare un valore di y il più vicino possibile al valore reale presente nel database.

Si cercherà quindi di minimizzare la funzione $f(\theta_0, \theta_1)$, che rappresenta l'errore quadratico medio tra i valori stimati e i valori reali.

Esempio:

Spostandoci in un caso semplificato: ho una retta in cui non lavoriamo con la generica, ma solo



con quelle che passano nell'origine.

$$h_0(x_1) = \theta_1 x_1$$

Supponiamo di avere 3 punti: (1,1), (2,2), (3,3).

Con la funzione:

Funzione iniziale

$$f(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

sostituisco:

$$f(\theta_1) = \frac{1}{6} ((1 - 1)^2 + (2 - 2)^2 + (3 - 3)^2)$$

Semplifico:

$$f(\theta_1) = 0$$

Se **spostassimo la retta**, noteremmo l'aumento dell'errore, e quindi un aumento della funzione $f(\theta_1)$, che rappresenta l'errore quadratico medio. E avrebbe un andamento a parabola (l'errore viene portato al quadrato, quindi non ci sarebbero negativi), con il minimo in corrispondenza del valore di θ_1 che meglio approssima i dati.

Parabola

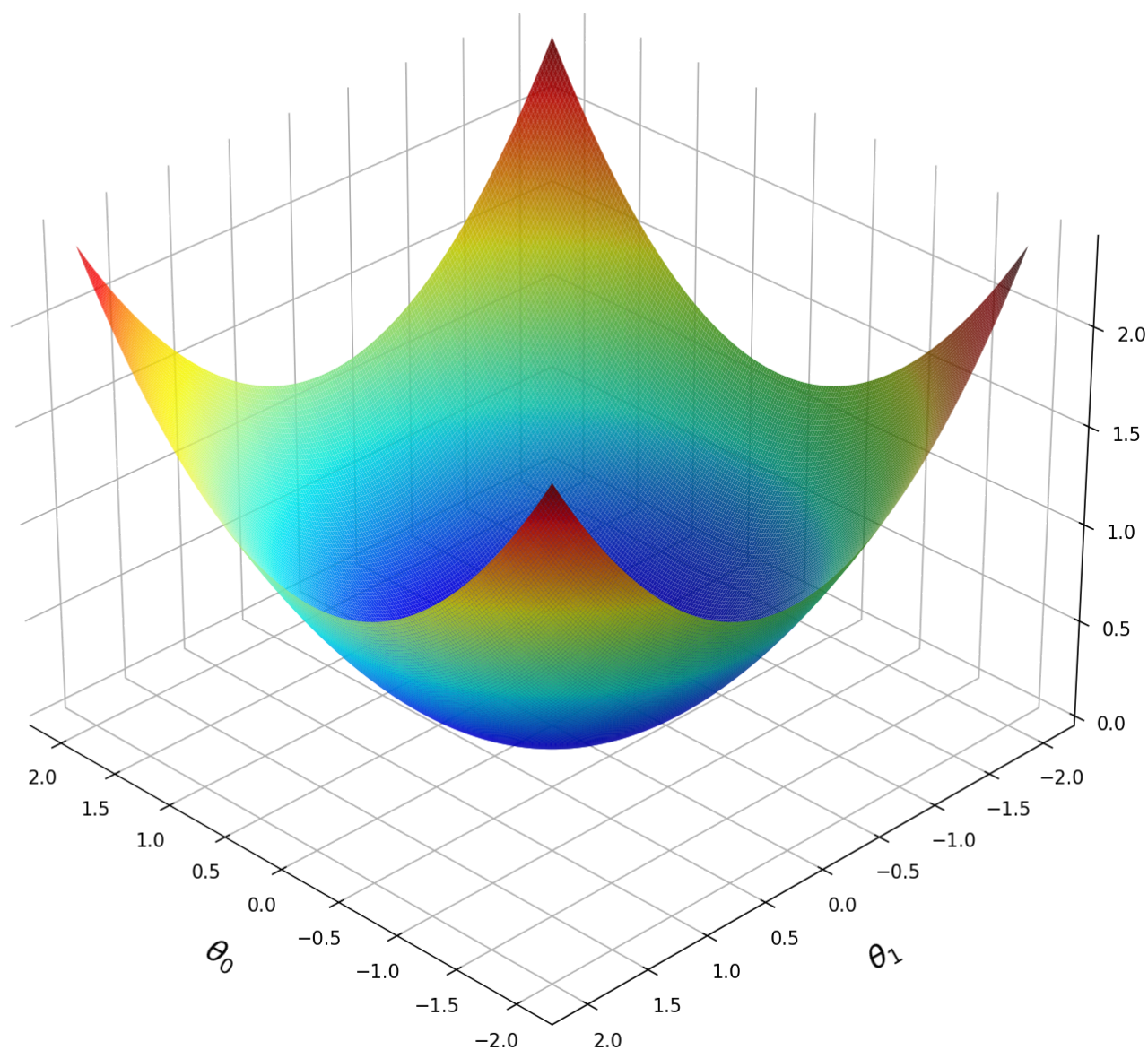
...

Per rappresentare tutte le possibili combinazioni di un grafico con **due variabili**, si potrebbe rappresentare un **grafico tridimensionale**, dove l'asse x rappresenta θ_0 , l'asse y rappresenta θ_1 , e l'asse z rappresenta il valore della funzione $f(\theta_0, \theta_1)$, che rappresenta l'errore quadratico medio.

Più il valore di $f(\theta_0, \theta_1)$ è **basso**, più la retta (o funzione) approssima bene i dati, e quindi più il punto corrispondente a quella combinazione di θ_0 e θ_1 è vicino al minimo della parabola.

Qualora avessimo più di una variabile, avremmo una funzione con un grafico tridimensionale.

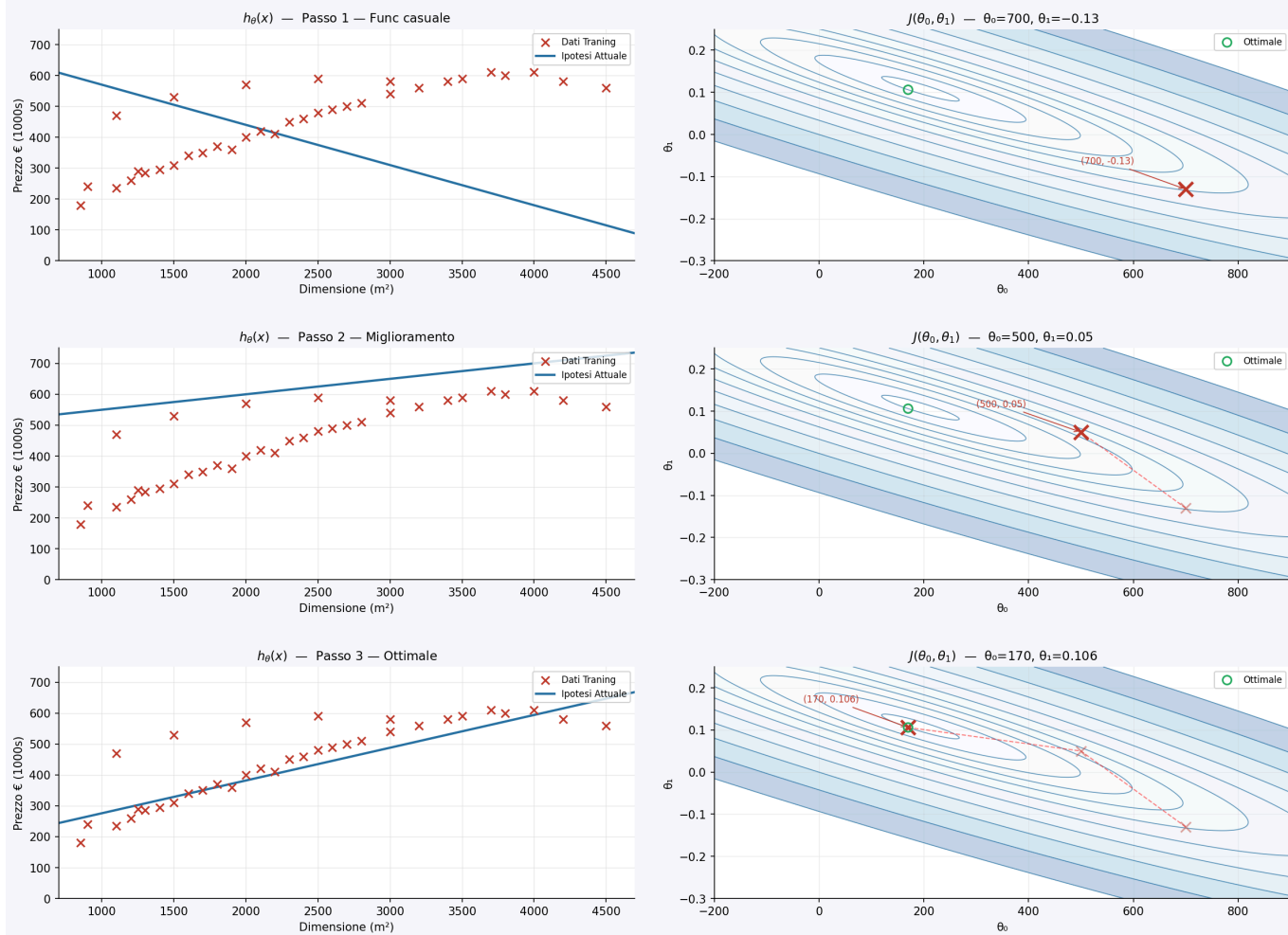
Un esempio: qualora volessimo rappresentare una **parabola**:



Se volessimo rappresentarlo in 2 dimensioni, dovremmo avere una funzione di livello, come le carte geografiche. Un asse del grafico rappresenterebbe θ_0 , l'altro θ_1 , e le curve rappresenterebbero i punti in cui la funzione $f(\theta_0, \theta_1)$ assume lo stesso valore.



Linee di Livello — Ipotesi e Precisione della funzione

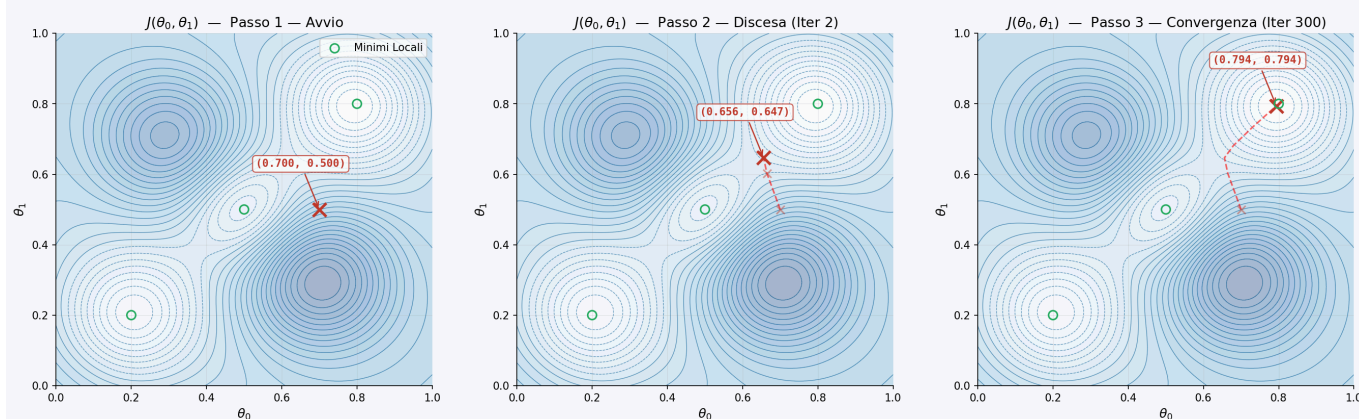


Possiamo andare in un punto a caso, osservare attorno, e muoverci verso un minimo relativo, fino a raggiungere il minimo globale. Questo processo è detto **discesa del gradiente**.

Il modo per calcolare tale discesa è quello di calcolare la derivata parziale della funzione $f(\theta_0, \theta_1)$ rispetto a θ_0 e θ_1 (vanno calcolate e confrontate **contemporaneamente**), in modo da capire in che direzione muoverci per ridurre l'errore.

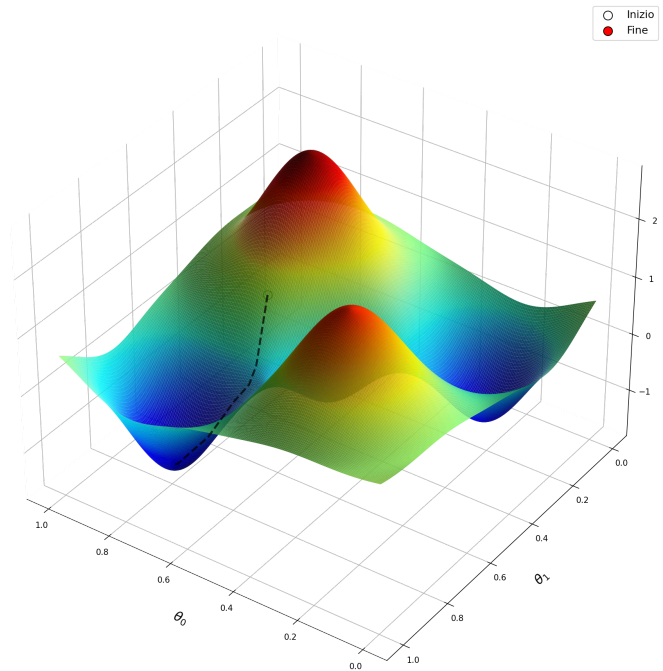
Un esempio, qualora avessimo una funzione più complessa, sia con la tecnica delle isoipse, sia con una rappresentazione tridimensionale:

Analisi Topologica: Discesa del Gradiente su Superficie Non Convessa





$$\theta_j = \theta_j - \alpha \frac{\partial J(\theta_0, \theta_1)}{\partial \theta_j} \quad \text{per } j = 0, 1$$

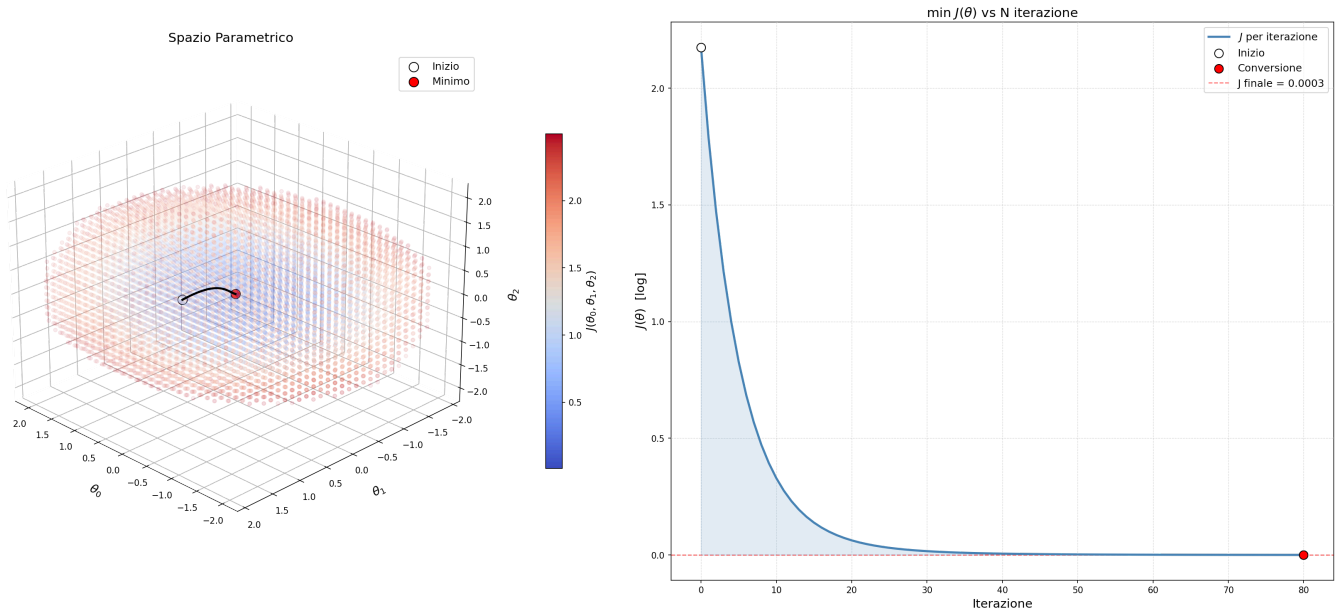


Questa procedura va eseguita sia per θ_0 che per θ_1 contemporaneamente, in modo da muoverci nella direzione che riduce l'errore.

Il processo di discesa del gradiente continua fino a quando non raggiungiamo un punto in cui la funzione $f(\theta_0, \theta_1)$ è minima (minimo globale idealmente, ma spesso si incontra un minimo locale, che se sufficientemente basso è accettabile), o fino a quando non raggiungiamo un numero massimo di iterazioni.

Con un numero sufficientemente elevato di variabili, la visualizzazione del processo diventa molto complesso, ma il concetto rimane lo stesso: stiamo cercando di trovare i valori di θ che minimizzano la funzione $f(\theta)$, che rappresenta l'errore quadratico medio tra i valori stimati e i valori reali.

Per comunque essere in grado di visualizzare l'andamento, valuta l'andamento di $J(\theta)$ nel tempo, cercando di farlo calare il più possibile.



Regressione lineare con più variabili

Qualora avessimo più di una variabile di input, una regressione lineare vista come prima non potrebbe essere applicata.

Se ad esempio avessimo:

Dimensione	Numero di Camere	Numero di Bagni	età	Prezzo (in k)
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...
x_1		x_2	x_3	x_4

Notazioni usate:

- n : numero di variabili di input
- $x^{(i)}$: valori di input per il i -esimo set di training. Esempio: $x^{(2)} = [1416, 3, 2, 40]$
- $x_j^{(i)}$: j -esima variabile di input per il i -esimo. Esempio: $x_3^{(2)} = 3$

Nel caso avessimo solo una variabile di input, la funzione di ipotesi era:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1$$

nel caso avessimo più variabili di input, la funzione di ipotesi diventa:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

**nel nostro caso:****esempio correlazione già calcolata**

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_4 x_4 \quad h_{\theta}(x) = 80 + 0.1x_1 + 0.01x_2 + 3x_3 - 2x_4$$

Nota come θ sia un **valore di bayes**.

Per comprendere la regressione, se inserissimo un nuovo set di valori al posto di $x_1, x_2, \dots, x_n$, troveremmo il valore di y stimato in base ai dati di training.

È anche utile per individuare in quale modo le varie variabili influiscano sul prezzo. In questo caso specifico notiamo che la metratura aumenta il prezzo con l'aumentare di essa, mentre l'età diminuisce il prezzo con l'aumentare di essa in maniera significativa.

Nei modelli più **complessi** ogni tanto alcuni parametri vanno **selezionati manualmente**, in modo da migliorare le prestazioni del modello.

Per modelli più semplici, una volta impostata la macchina correttamente, è sufficiente eseguire il codice per ottenere la correlazione (con risultati solitamente inferiori ai modelli raffinati a mano).

Come calcolare la discesa del gradiente con più variabili?

Nel caso avessimo 4 variabili di input come sopra:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

Questo perché i prodotti tra matrici sono molto più efficienti dei cicli *for*.

$$h_{\theta}(x) = \theta^T x$$

la funzione di costo diventa:

$$f(\theta_0, \theta_1, \dots, \theta_n) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

e valutiamo (come per una lineare a due variabili), la discesa del gradiente:

$$\min_{(\theta_0, \theta_1, \dots, \theta_n)} J(\theta_0, \theta_1, \dots, \theta_n)$$

Bisogna fare attenzione qualora non si avessero dati non omogenei, o con scale molto diverse, perché la funzione di costo funziona bene solo se i dati sono **normalizzati**.

Nel caso non venissero normalizzati, la funzione di costo diventerebbe una **valle molto stretta**, ed è un fenomeno che rallenta significativamente la discesa del gradiente.

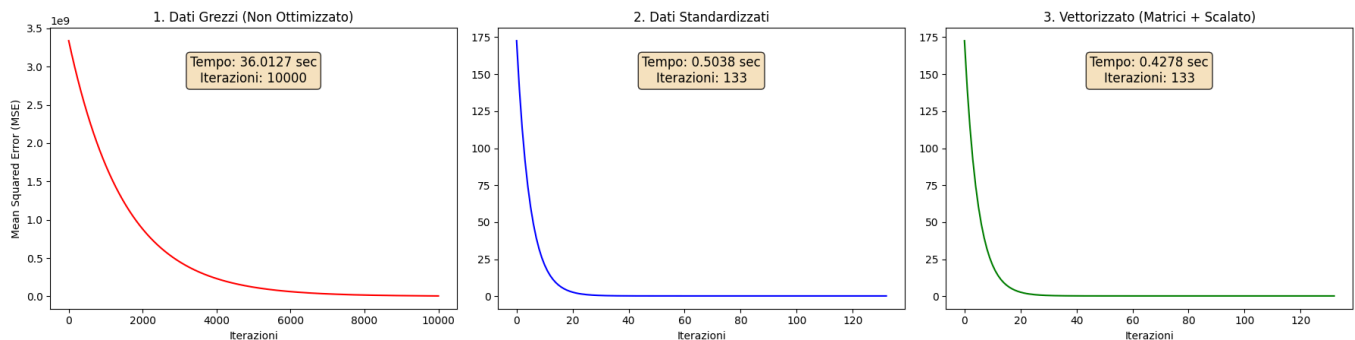


Normalizzazione Mediana Normalizzazione Min-Max

$$x_j = \frac{x_j - \mu_j}{\sigma_j}$$

$$x_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$$

Confronto Metodi Gradient Descent: Curva di Apprendimento (MSE vs Iterazioni)

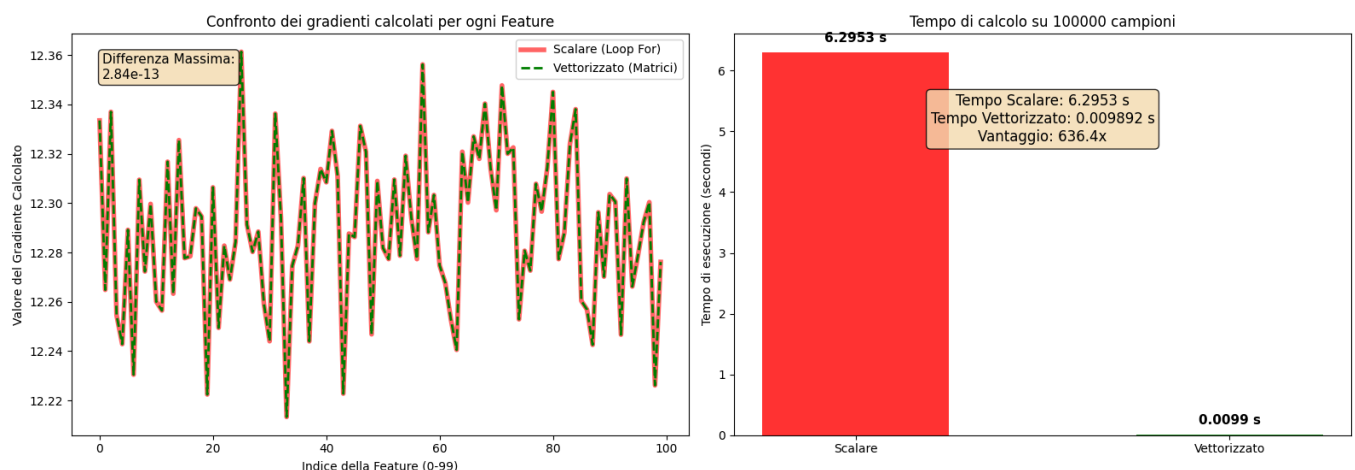


La versione non normalizzata impiega un numero di iterazioni molto più elevato e quindi un tempo di addestramento molto più elevato, rispetto alla versione normalizzata. La versione normalizzata con l'uso di prodotto tra matrici, utilizza il medesimo numero di iterazioni, ma con un tempo relativamente molto più basso di tempo utilizzato. Questo avviene perché i for sono molto più lenti dei prodotti tra matrici, soprattutto in questo caso che usiamo dei for in Python, mentre il calcolo tra matrici è gestito da librerie ottimizzate come NumPy, che si basano su C

Notare come nel primo caso si raggiunga un numero di iterazioni pari a 10000 (il numero massimo impostato nella simulazione), significante che non ha mai raggiunto un punto di minimo

Per visualizzare la differenza tra loop e prodotti tra matrici, prendere in considerazione il seguente grafico, dove è stato usato un dataset molto più grande e vario, con 100000 esempi di training e 100 variabili di input:

Confronto Gradient Descent: Precisione vs Performance



★ (Notare come vi sia una minuscola differenza (alto a sinistra) nei valori tra loop e matrice, dovuta a come Python e NumPy gestiscono i numeri in virgola mobile, ma che è talmente piccola da essere **trascurabile** (in questo caso 2.84×10^{-13}).)★



Un'altra tecnica per minimizzare la funzione di costo è quella di traslare i dati, in modo da spostare il minimo della funzione di costo in un punto più vicino all'origine, e quindi ridurre il numero di iterazioni necessarie per raggiungere il minimo.

Regressione polinomiale

Quando ci sono dei dati che non sono lineari, è necessario aumentare il grado del polinomio per poter approssimare meglio i dati.

Come si capisce che coefficienti usare?

Il numero di ipotesi che si potrebbero generare è pressoché infinito, contro quelli della regressione lineare, che è limitato a n variabili di input.

Per un numero delle feature molto basso si ci potrebbe limitare a disegnarlo su un grafico, e quindi scegliere la funzione che meglio approssima i dati.

Nel caso ne avessimo più di una, questo principio non è applicabile.

Le funzioni polinomiali possono essere tuttavia **mappate** in delle sotto funzioni lineari che possono approssimare sufficientemente i dati, e quindi essere risolte con una regressione lineare con più variabili.

Funzione iniziale

Funzione mappata

$$f(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 \quad f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

Coefficienti di Pearson

$$p = \frac{1}{N-1} \frac{\sum_{i=1}^n (x_i - \bar{x})}{\sigma_x} \frac{(y_i - \bar{y})}{\sigma_y}$$

$$\sigma_x = \sqrt{\frac{1}{N-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Qualora i dati siano fortemente correlati (al aumentare di uno aumenta anche l'altro), il coefficiente sarà tendente a 1, mentre se sono negativamente correlati (se uno aumenta, l'altro scende), sarà tendente a -1. Se invece non sono correlati, sarà tendente a 0.

Questo è utile qualora avessimo un numero molto elevato di features, dove usarle tutte diventerebbe impraticabile, quindi con la correlazione individuiamo quelle più correlate con l'output e quindi usiamo solo quelle per l'addestramento. In alternativa, se due features sono



fortemente correlate tra di loro, possiamo eliminarne una, in quanto fornisce informazioni simili all'altra, e quindi non è necessario usarle entrambe.



Training set, test

Quando abbiamo un dataset su cui addestrare la macchina, va diviso in due parti:

- **Training set:** è la parte del dataset su cui la macchina viene addestrata, e quindi su cui impara a riconoscere le caratteristiche che distinguono un elemento da un altro. Solitamente rappresenta il 60-80% del dataset totale.
- **Test set:** è la parte del dataset su cui la macchina viene testata, e quindi su cui si valuta la capacità della macchina di generalizzare, ovvero di riconoscere le caratteristiche che distinguono un elemento da un altro anche su dati che non ha mai visto prima. Solitamente rappresenta il 20-40% del dataset totale.

È **FONDAMENTALE** che il test segua la medesima normalizzazione del training set, altrimenti la macchina non sarebbe in grado di predire i valori. Un esempio di classico errore è quello di usare min-max per normalizzare, e poi cambiare i valori di min max per normalizzare il test. Vanno usati i medesimi valori usati per normalizzare il training set.

Una volta ottenuti i risultati dei test, per poterli valutare correttamente, è necessario contronormalizzarli, in modo da portarli di nuovo a valori comprensibili e reali.



Logistic Regression

qualora avessimo dei dati non rappresentabili con una retta, usiamo la formula:

$$g(z) = \frac{1}{1 + e^{-z}}$$
$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Dividiamo quindi i risultati in 1 e 0, dove 1 rappresenta un elemento positivo, e 0 rappresenta un elemento negativo.

Qualora il valore sia maggiore di 0.5, lo consideriamo positivo, altrimenti lo consideriamo negativo.

Cost Function

$$\begin{cases} h_{\theta}(x) \geq 0.5 & \text{se } y = 1 \\ h_{\theta}(x) < 0.5 & \text{se } y = 0 \end{cases}$$

e viene definita con la seguente funzione:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x), y)$$

dove:

$$\text{Cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

Decision Boundary

È una retta che divide i valori accettati da quelli non accettati. Viene definita dalla formula:

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$
$$\text{con: } \theta = [\theta_0, \theta_1, \theta_2]^T, \quad \mathbf{x} = [1, x_1, x_2]^T$$

fai esempi

Decision Boundary non lineare



Confusion Matrix - Matrice di confusione

Vengono utilizzati per determinare la qualità di un modello.

	Predetti Positivi	Predetti Negativi
Effettivi Positivi	Vero Positivo (TP)	Falso Negativo (FN)
Effettivi Negativi	Falso Positivo (FP)	Vero Negativo (TN)

- I Falsi Positivi sono chiamati **Type I error**.
- I Falsi Negativi sono chiamati **Type II error**

È una matrice che serve a valutare un classificatore addestrato.

Il al modello vengono forniti dei dati di test, si osserva il valore di output e si inserisce addebitamente nella matrice in relazione al valore reale.

Per ottenere risultati accurati, è importante che i valori di test abbiano delle etichette, in modo da poter automatizzare il processo di inserimento nella matrice. Con l'aumentare dei dati di test, il valore della matrice si avvicina a quello reale.

Solitamente si lascia il modello nottetempo - o per più giorni - a creare diversi classificatori con diversi test, per poi determinare il migliore grazie al confronto dei risultati.

Per valutare rapidamente una matrice di confusione, si tiene conto della seguente formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Questo però funziona **SOLO** se i dati sono bilanciati, ovvero se il numero di positivi e negativi è simile. Ad esempio già con 51/49 è meno affidabile, e con 90/10 è praticamente inutile. Ci sono interi sistemi che sono dedicati a individuare eventuali anomalie.

Deduzione della performance del modello da una matrice di confusione

Uso la formula per calcolare la precisione, che rappresenta la percentuale di predizioni positive corrette rispetto al totale delle predizioni positive:

$$P = \frac{TP}{TP + FP}$$

Uso la formula per calcolare il richiamo (**recall**), che rappresenta la percentuale di predizioni positive corrette rispetto al totale dei positivi effettivi:

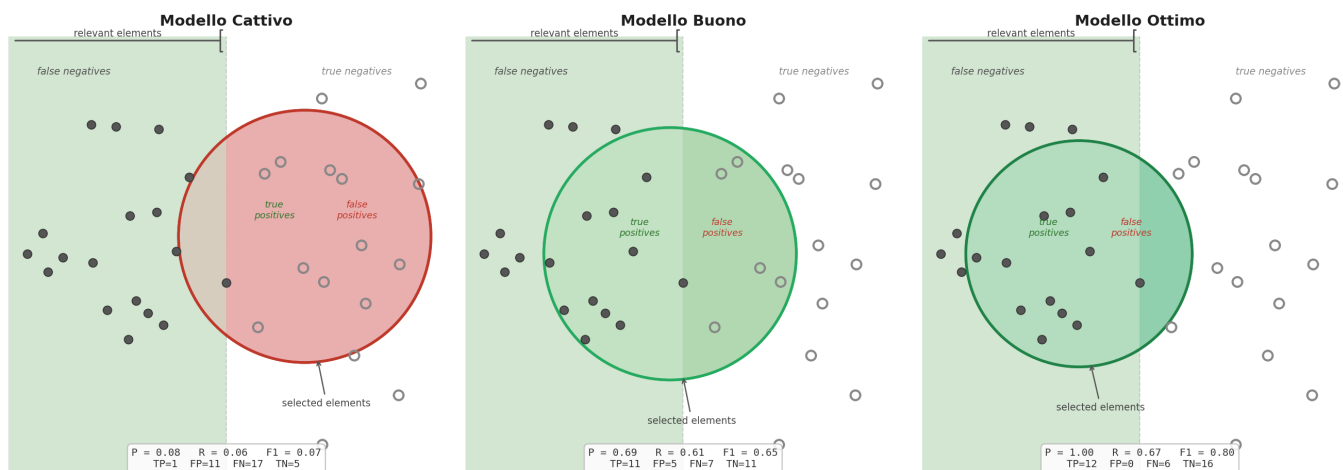


$$R = \frac{TP}{TP + FN}$$

Si usa poi la formula per calcolare il punteggio F1, che rappresenta la media armonica tra precisione e richiamo, e fornisce una misura complessiva della performance del modello:

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Precisione, Richiamo e F1 — Confronto tra Modelli



- I pallini scuri sono valori risultati positivi, mentre i pallini chiari sono valori risultati negativi.
- I pallini scuri all'interno del cerchio sono i veri positivi, mentre quelli chiari all'interno del cerchio sono i falsi positivi (valori negativi che il modello ha classificato come positivi).

Confusion Matrix con più classi

Fino a ora abbiamo osservato solo dei classificatori binari, che classificano i dati in due classi (positivi e negativi).

Nel caso avessimo più classi, la matrice di confusione diventa più complessa, in quanto dobbiamo considerare tutte le possibili combinazioni di classi.

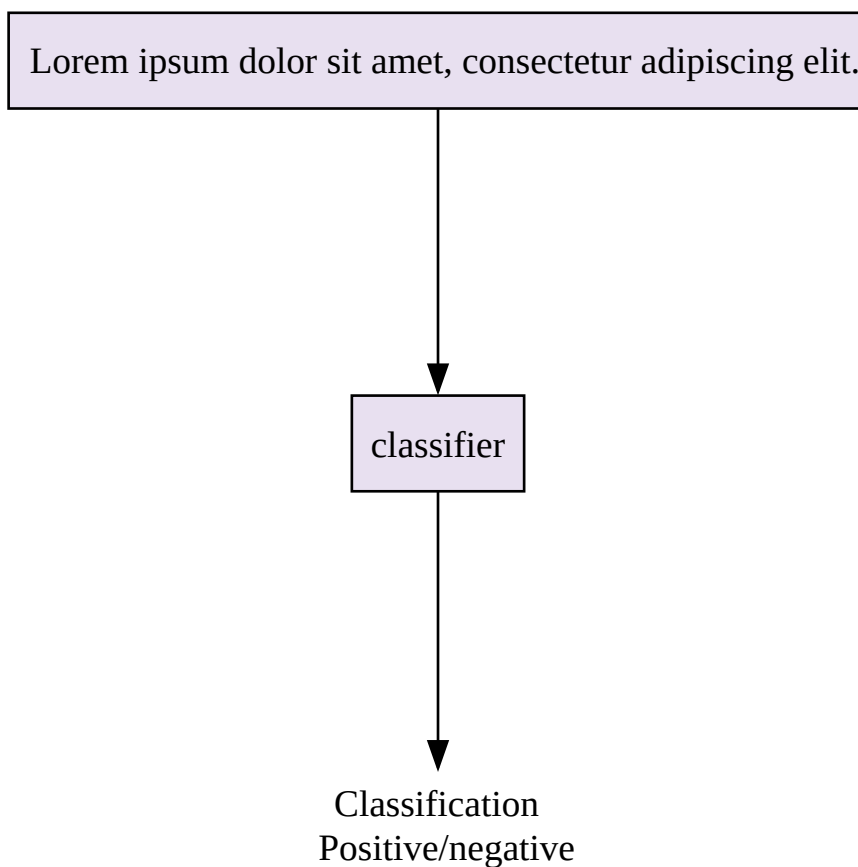
Ad esempio, se avessimo 3 classi (A, B, C), la matrice di confusione sarebbe la seguente:

	Predetti A	Predetti B	Predetti C
Effettivi A	TP_A	FP_BA	FP_CA
Effettivi B	FP_AB	TP_B	FP_CB
Effettivi C	FP_AC	FP_BC	TP_C



Text Data

I modelli che lavorano con Text Data, come gli LLMs, consentono di classificare i dati in base al loro contenuto testuale.



A differenza dei modelli visti in precedenza, ci sono voluti moltissimi anni per potere classificare se una frase fosse negativa o positiva, perchè un testo non ha un chiaro input come un numero, che o è positivo o è negativo, ma una serie di caratteri (quindi codici ascii) che nel loro complesso determinano se una frase è positiva o negativa.

PreProcessamento

Per ridurre la complessità del testo, si inizia con rimuovere o semplificare varie parole, in modo da ridurre il numero di variabili di input.

Alcuni esempi di pre-processamento sono:

- Parole con diversi contenuti semantici, ma con la stessa radice, vengono ridotte alla radice. Ad esempio "correre", "corro", "corriamo" diventano tutti "corr".
- Uso di maiuscole e minuscole: "Ciao" e "ciao" diventano entrambi "ciao".

La differenza tra un modello passato attraverso un pre-processing e uno senza tale operazione, è enorme.



Lemmatizzazione

L'idea è come quella di ridurre la parola alla radice, come un vocabolario, in modo da ridurre il numero di variabili di input.

La differenza tra il metodo visto prima (detto **stemming**), è che in questa maniera si preservano le differenze, ma le performance non migliorano significativamente.

Stop Words

Vengono rimossi gli articoli e la punteggiatura è un altro modo di ridurre la complessità del testo, tuttavia nuovi studi mostrano che questo metodo impatta negativamente sulla qualità del modello, con un aumento delle prestazioni non giustificabili in comparazione alla perdita di qualità.

Vengono spesso salvate come una lista di parole, definita da linguisti per ogni lingua, che vengono sistematicamente rimosse dal testo prima di essere processato dal modello.

Espressioni regolari

Tutti gli elementi comuni, come struttura html, link, email, numeri di telefono, vengono rimossi o sostituiti con dei token specifici, in modo da ridurre la complessità del testo e preservare solo le informazioni rilevanti per il modello.

Esempi di pre-processamento

Testo originale	Testo pre-processato
I have been impressed with Chavez's stance against globalisation for sometime now, but it wasn't until I saw the film at the Amsterdam documentary international film festival that I realize what he has really achieved. This film tells the story of coup/conspiracy by Venezuela's elite, the oil companies and oil loving corrupt western governments, to remove democratically elected president Chavez, and return Venezuela back to a brutal dictatorship. This film is must for anyone who believes in freedom and justice, and is also a lesson to the rest of world ! I commend the people of Venezuela for taking matter into their own hands, and saving their country from the likes of Halliburton and the Bush regime.	I have been impress with chavez stanc against globalis for sometim now but it wasn't until i saw the film at the amsterdam documentari intern film festiv that i realiz what he has reall achieved this film tell the stori of coup/conspiraci by venezuela elite the oil compani and oil love corrupt western governments to remov democrat elect presid chavez and return venezuela back to a brutal dictatorship. this film is must for anyon who believ in freedom and justice and is also a lesson to the rest of world! i commend the peopl of venezuela for take matter into their own hands and save their countri from the like of halliburton and the bush regime



Bag of Word Approach

Possiamo pensare di creare un vocabolario, dove ogni nuova parola che incontriamo viene aggiunta ad una lista (**bag of words**), e ogni parola ha il proprio indice.

Ogni volta che viene trovata una parola già presente nella lista, viene sostituita con il suo indice, altrimenti viene aggiunta alla lista e viene assegnato un nuovo indice.

Esempio:

Testo originale	Bag of Words
I have been impressed with Chavez's stance against globalisation for sometime now, but it wasn't until I saw the film at the Amsterdam documentary international film festival that I realize what he has really achieved. This film tells the story of coup/conspiracy by Venezuela's elite, the oil companies and oil loving corrupt western governments, to remove democratically elected president Chavez, and return Venezuela back to a brutal dictatorship. This film is must for anyone who believes in freedom and justice, and is also a lesson to the rest of world ! I commend the people of Venezuela for taking matter into their own hands, and saving their country from the likes of Halliburton and the Bush regime.	22 44 3 916 1077 883 ... 370 1699 790 1822 1831 883 431 1171 ... 794 1002 1893 85 592 1676 238 162 85 688 945 1663 1120 ... 1062 1699 375 1162 479 1893 1510 799 1182 1237 85 1895 1440 ... 1547 181 1699 1758 1896 688 ... 1676 992 961 ... 1477 71 530 1699 1898

Un gran problema di questo sistema, è che è molto variabile

Viene risolto con la creazione di un vettore di lunghezza pari al numero di parole presenti nel vocabolario, indicizzato con tutti valori interni pari a 0, che poi vengono incrementati di 1 ogni volta che viene trovata la parola corrispondente.

Una volta creato il vettore, si controlla la frequenza di ciascun valore (e quindi parola) il modello lo usa per capire di quale argomento si stia parlando

Il vantaggio è che quando ho creato un vettore sufficientemente grande, posso rappresentare qualsiasi testo.

LIMITAZIONI:

- Essendo tutto basato sulla frequenza delle parole, due frasi con le stesse parole, ma con un significato completamente diverso, verrebbero rappresentate nello stesso modo, e quindi il modello non sarebbe in grado di distinguere tra le due frasi.

Esempio:

- "Il gatto è sul tavolo"
- "Il tavolo è sul gatto"



TF - IDF (Term Frequency - Inverse Document Frequency)

Dato un documento d e una parola w

$$TfIdf(w, d) = tf \cdot \left(\ln \left(\frac{N + 1}{N_w + 1} \right) + 1 \right)$$

- N : numero totale di documenti
- N_w : numero di documenti che contengono la parola w
- tf : Term Frequency, ovvero la frequenza della parola w nel documento d

è importante tenere conto che se una parola ha un valore molto elevato, la sua importanza è bassa, perchè usata in molti documenti, mentre se una parola ha un valore molto basso, la sua importanza è alta, perchè usata in pochi documenti e quindi rappresenta un indicatore di specializzazione.

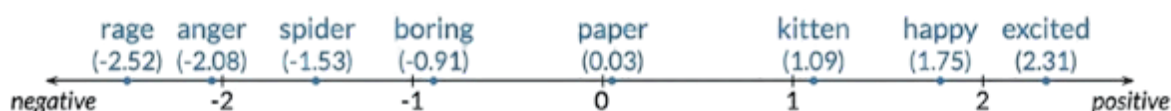
Word Representations

I metodo precedenti affidavao un vettore alla frase che descriveva in maniera generale il testo, ma non consentiva di andare nel particolare. Con il metodo delle word representations, invece, viene creato un vettore per ogni parola, che rappresenta il significato della parola in base al contesto in cui viene usata. In modo da poter trattare nel dettaglio il testo, e se fosse necessario generalizzare sucessivamente.

Vi sono molteplici metodi per creare le word representations, ma molti con diverse limitazioni come:

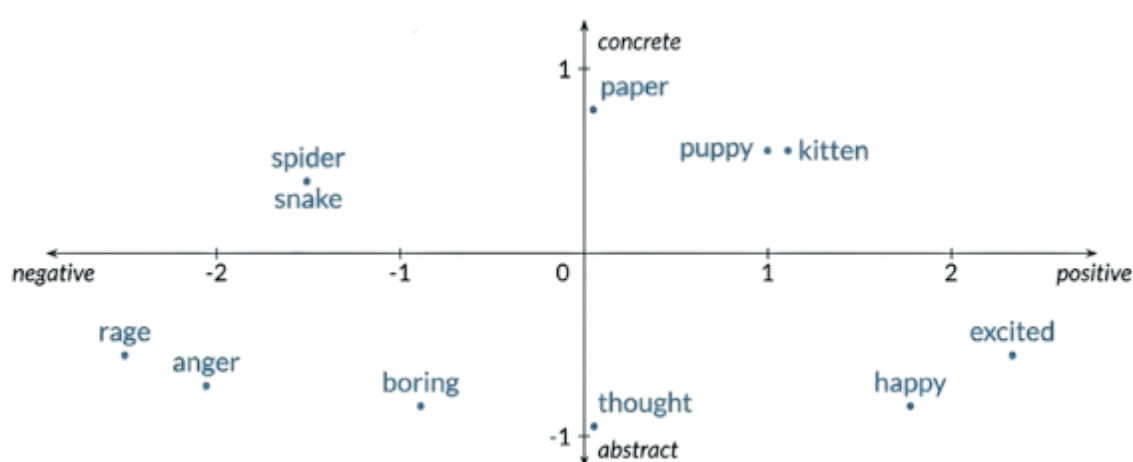
- **Integers**: ogni parola viene rappresentata con un numero intero, ma questo metodo non consente di rappresentare la relazione tra le parole, e quindi non è molto efficace.
- **One-Hot Encoding**: ogni parola viene rappresentata con un vettore di zeri e un uno, ma questo metodo non consente di rappresentare la similarità tra le parole (oltre che essere estremamente grande dimensionalmente).
- **Word Embeddings**: ogni parola viene rappresentata con un vettore di numeri reali, che rappresentano il significato della parola in base al contesto in cui viene usata. Questo metodo consente di rappresentare la similarità tra le parole, e quindi è molto efficace. (un esempio, "felice" e "contento" avrebbero vettori molto simili, mentre "felice" e "triste" avrebbero vettori molto diversi).

Word Embeddings



Come accennato prima, ad ogni parola viene assegnato un valore reale, che rappresenta il significato della parola in base al contesto in cui viene usata.

Questo principio consente classificazione basica, perchè usa un solo asse, qualora volessimo un modello più complesso, verrebbero aumentati il numero di assi, in modo da rappresentare più caratteristiche della parola.



questo principio ci consente di rappresentare semanticamente le parole, anche in base al contesto in cui vengono usate.

Per realizzare questi modelli, è necessario automatizzare il processo di creazione dei vettori. Questo processo per gli esseri umani è semplice ed intuitivo (con il difetto di essere molto lento), ma per una macchina è molto complesso.

Se volessimo aumentare il numero di assi, anche per gli esseri umani diventa molto complesso eseguire la classificazione.

Questi sistemi lavorano molto a livello posizionale, ovvero cercando di trovare la parola più vicina a quella data, in modo da poterla classificare in base al contesto.

ad esempio, se fornissi la frase: *"rosso di sera..."* verrebbe naturale completarla con *"...bello tempo si spera"*.

Ma come viene realizzato questo processo?

All'inizio ogni parola viene rappresentata in maniera casuale, quindi la macchina inizia a cercare di vedere quale parole tendono ad essere più vicine tra di loro, iniziando a notare trovare pattern.



Ogni volta che trova due parole vicine, leggendo un testo, le avvicina, poco alla volta, in modo da creare dei cluster di parole simili tra di loro, e quindi rappresentare semanticamente le parole.

Questo processo di solito non è in 2 dimensioni come visto prima, ma da 300 a 500 dimensioni (ovvero 300-500 assi), in modo da rappresentare più caratteristiche della parola, e quindi creare dei cluster più precisi, dopo aver analizzato milioni o miliardi di parole.

Nei modelli più complessi (come gli LLMs), la macchina stessa decide e crea i propri assi, in modo da rappresentare le caratteristiche che ritiene più rilevanti per la classificazione, e quindi creare dei cluster più precisi.



Holdout method and Regularization

è un concetto molto importante, soprattutto a livello tecnico. Ovvero l'idea di come gestire il processo di creazione di un modello, da inizio a fine.

Un esempio è se volessimo creare un modello per analizzare le immagini di un aeroporto, per individuare i volti per motivi di sicurezza.

Si necessita quindi di un dataset con il quale si addestra un modello, si inizia con:

- **Original set:** è il dataset originale, che contiene tutte le immagini dell'aeroporto, con i volti e senza i volti.
 - **Training set:** è il dataset che viene usato per addestrare il modello, e quindi contiene una parte delle immagini dell'aeroporto, con i volti e senza i volti. Solitamente rappresenta il 60-80% del dataset totale.
 - **Training set (di addestramento):** viene ottenuto dividendo il training set in due parti, una parte viene usata per addestrare il modello, e l'altra parte viene usata per testare il modello durante il processo di addestramento, in modo da poter valutare la performance del modello e quindi decidere quando fermare l'addestramento. Solitamente rappresenta il 70-80% del training set totale.
 - **Validation set:** viene ottenuto dividendo il training set in due parti, una parte viene usata per addestrare il modello, e l'altra parte viene usata per testare il modello durante il processo di addestramento, in modo da poter valutare la performance del modello e quindi decidere quando fermare l'addestramento. Solitamente rappresenta il 20-30% del training set totale.
 - **Test set:** è il dataset che viene usato per testare il modello, e quindi contiene una parte delle immagini dell'aeroporto, con i volti e senza i volti. Solitamente rappresenta il 20-40% del dataset totale. È fondamentale che una volta avvenuta la divisione, questi dati non vengano toccati se non per la fase finale

La fase più lunga è quella di addestramento, dove si provano diversi parametri e metodi per creare il modello, dove viene continuamente testato con il validation set, in modo da poter valutare la performance del modello e quindi decidere quando fermare l'addestramento.

Ma come mai abbiamo 3 set diversi anziché 2 direttamente?

Se addestrassimo il modello direttamente sul test set, avremmo un modello eccellente per i dati su cui abbiamo a disposizione, ma non siamo certi se sia in grado di generalizzare su dati mai visti. Il termine tecnico per questo fenomeno è **overfitting**, ovvero il modello si è adattato troppo ai dati di addestramento.

Nella divisione è importante usare delle tecniche per ridurre i bias:

Ad esempio, nel corso della giornata, la luce del sole cambia, come la quantità delle persone,



quindi è importante che il test set contenga la più possibile varietà di dati, in modo da poter valutare la performance del modello con più varietà nei dati.

Nel primo training set, viene usato il **cross validation** per evitare questo fenomeno.

K-Fold Cross Validation

L'idea è quella di dividere il training set in k parti, di cui solo uno sarà usato per validare il modello, mentre gli altri $k-1$ saranno usati per addestrare il modello.

Questo processo viene ripetuto per tutte le k parti, ogni volta scambiando la posizione del validation set (scorrendo da $k=0$ a $k=k$), in modo da poter valutare la performance del modello con più varietà nei dati, e quindi ridurre il bias.

I dati da un addestramento all'altro non cambiano, ma cambia solo quale porzione viene usata per validare il modello, e quale porzione viene usata per addestrare il modello.

porzione	Round 1	Round 2	...	Round k
1	validation set	training set	...	training set
2	training set	validation set	...	training set
3	training set	training set	...	training set
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
k	training set	training set	...	validation set

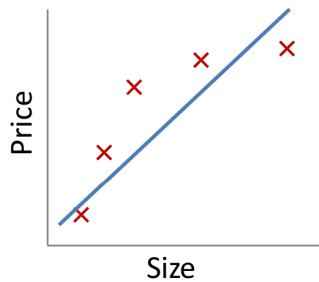
Questa operazione è parecchio costosa dal punto di vista computazionale, perchè aumenta di k volte il tempo di addestramento, ma è molto efficace per ridurre il bias, e quindi ottenere un modello più generalizzabile.

Con modelli più complessi, come gli LLMs, non è possibile usare questa tecnica, perchè un singolo addestramento è già di diversi mesi, quindi effettuarne k impiegherebbe diversi anni, e gli investitori non sarebbero contenti.



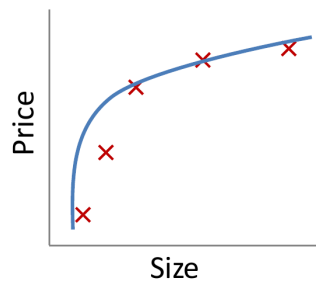
Regularization

Regressione lineare 2D e classificazione



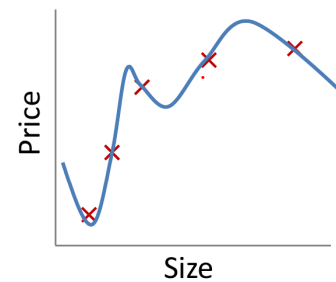
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Underfit



$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

Good

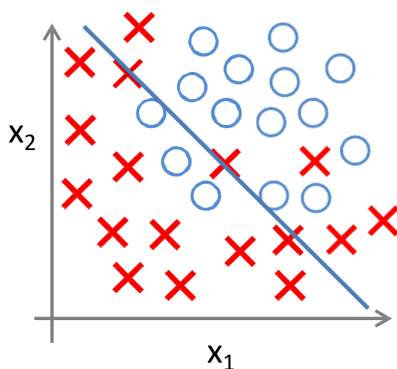


$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Overfit

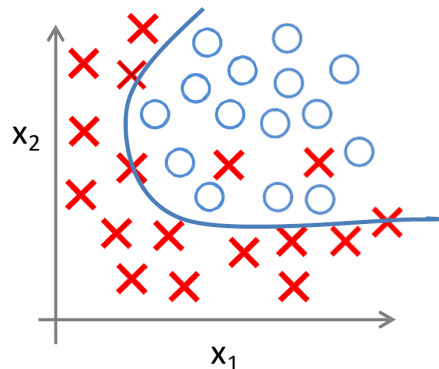
Nel caso di **underfitting**, nella regressione lineare la funzione non si adatta ai dati, e quindi non è in grado di predire i valori, perchè è troppo semplice, e quindi non è in grado di generalizzare su dati mai visti (o dati già visti).

Nel caso di **overfitting**, nella regressione lineare la funzione si adatta perfettamente ai dati, ma appena mostriamo un nuovo dato leggermente diverso, la polinomiale non è in grado di predire il nuovo valore, perchè è troppo complessa, e quindi non è in grado di generalizzare su dati mai visti.



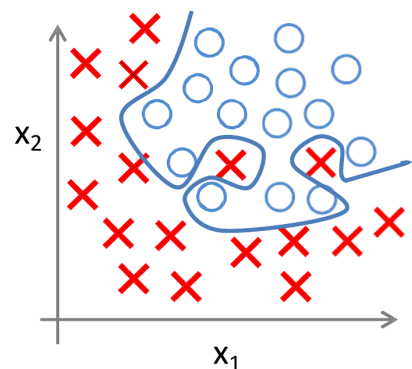
$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

Underfit
(High Bias)



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

Good



$$h_{\theta}(x) = g(\theta_0 + \theta_1 x + \theta_2 x_2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

Overfit
(High Variance)



nel caso di **underfitting**, la classificazione è troppo semplice, e quindi non è in grado di distinguere tra i dati, e quindi non è in grado di generalizzare su dati mai visti (e neanche su dati che ha già visto).

Nel caso di **overfitting**, è un problema perchè in pochissimo spazio, un dato passerebbe dal essere da una classe all'altra, ad una distanza molto breve, anche se i dati sono molto vicini tra di loro, e quindi il modello non sarebbe in grado di generalizzare su dati mai visti.

In uno spazio bidimensionale è facile visualizzare questi fenomeni, ma con più dimensioni diventa molto complesso se non impossibile visualizzarli, e quindi bisogna nuovamente affidarsi al calcolo matematico per poterli individuare e eventualmente risolverli, tipicamente confrontando le performance sul training e sul validation set.

- Se noto che aumentando la complessità del modello, le performance sul training set migliorano, ma le performance sul validation set peggiorano, allora è un chiaro segnale di overfitting, e quindi è necessario ridurre la complessità del modello.
- Se noto che aumentando la complessità del modello, le performance sul training set migliorano, ma le performance sul validation set migliorano (anche leggermente), allora potrebbe essere un segnale di underfitting, e quindi è necessario aumentare la complessità del modello.
- viceversa

Metodi per risolvere il problema in molteplici dimensioni:

- Ridurre il numero di variabili di input
 - Selezionare manualmente le variabili più rilevanti
 - Usare modelli di selezione (**PCA**)
- Regularizzazione (**Regularization**)
 - Tenere tutte le variabili, ma ridurre i loro coefficienti, in modo da ridurre la complessità del modello, e quindi evitare l'overfitting.
 - ...

Per affrontare entrambi questi problemi, si usa la **regularization**

In linea di massima, per evitare l'overfitting, il numero di variabili deve essere inferiore, di almeno un ordine di grandezza, al numero di esempi di training, in modo da poter generalizzare su dati mai visti.

Regularization: Ridge regression

Viene cambiato il processo di minimizzazione della funzione di costo:



$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$
$$\min_{(\theta)} J(\theta)$$

Ma come funziona?

Si inserisce dentro la funzione di costo un cosiddetto **termine di regolarizzazione**, che va a giocare sui valori dei parametri che ho all'interno della funzione di costo.

vediamo alcuni esempi estremi:

esempio	formula	note
$\lambda = 0$	$\frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 0 \right]$	non influenza il modello
$\lambda \rightarrow \infty$	$\frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \infty \cdot \sum_{j=1}^n \theta_j^2 \right]$	il costo diventa enorme, quindi il gradient descent non converge e quindi cerca di stare più vicino a 0 i parametri, ma in questo caso il modello è troppo semplice, perchè non



esempio	formula	note
---------	---------	------

impara
nulla

quindi λ più viene aumentato, e più viene ridotta la capacità del modello di adattarsi ai dati

Regularization: Logistic regression

Un modello nota che la presenza di un determinato elemento è un indicatore di una classe, per esempio se un modello inizia a trovare la parola Trump in un testo e lo classifica come un testo politico, allora la presenza di quella parola diventa un indicatore di quella classe, e quindi il modello si adatta a quella parola, e quindi se trova quella parola in un testo, lo classifica come politico.

Questo processo avviene su molteplici testi, e quindi il modello inizia ad adattarsi a quelle parole, e quindi se trova quelle parole in un testo, lo classifica come politico.

Tuttavia, con il tempo, la sua presenza nei testi di politica inizierà a svanire (per esempio tra 30 anni, non sarà più in politica) e quindi il modello non sarà più in grado di classificare correttamente i testi politici, perchè si è adattato troppo a quelle parole, e quindi non è in grado di generalizzare su dati mai visti.

Un buon modello invece, sarebbe in grado di trovare diversi indicatori di quella classe, attribuendo a ciascuno di essi un peso simile (o per lo meno non uno che abbia il completo controllo) quindi anche se un indicatore svanisce *-anche se è il più importante-* è comunque in grado di operare (notare come senza regolarizzazioni sia un indicatore ², quindi se svanisse il modello ne risentirebbe molto)

Il problema della minimizzazione:

$$J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2 \right]$$

$$\text{usiamo } c = \frac{1}{\lambda}$$

$$J(\theta) = \frac{1}{m} \left[c \sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \frac{1}{2} \sum_{j=1}^n \theta_j^2 \right]$$



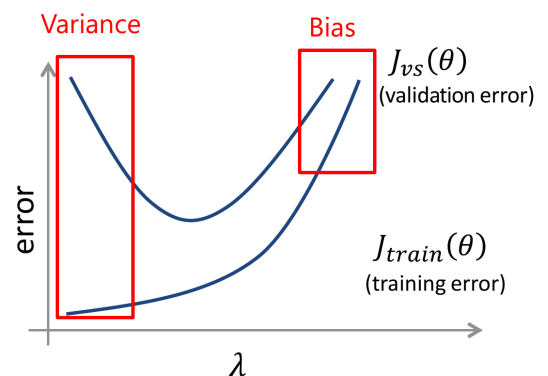
Bias-Variance

Se utilizzassimo la funzione di costo vista in precedenza:

$$J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$$= \frac{1}{2m} \left[\sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

Dobbiamo trovare un compromesso tra i due termini, in modo da ottenere un modello che sia in grado di generalizzare su dati mai visti, ma che allo stesso tempo non sia troppo semplice, e quindi non sia in grado di imparare nulla.



$$J_{train}(\theta) = \frac{1}{m_{train}} \left[\sum_{i=1}^{m_{train}} \text{Cost}(h_{\theta}(x), y) \right]$$

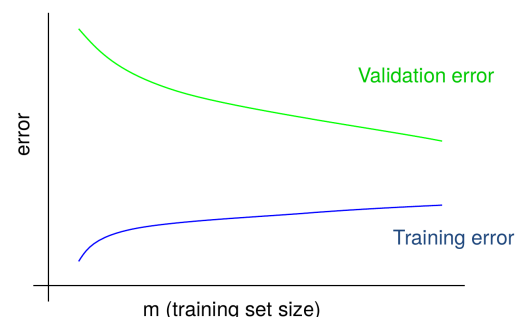
$$J_{vs}(\theta) = \frac{1}{m_{vs}} \left[\sum_{i=1}^{m_{vs}} \text{Cost}(h_{\theta}(x), y) \right]$$

Learning curve

Cercano di farci vedere il comportamento del sistema in base alla quantità di dati di addestramento.

Vengono eseguiti una serie di addestramenti "a pezzetti", calcolando ogni volta due valori:

- **validation error:** è l'errore che ottengo quando testo il modello con il validation set, e quindi mi dà un'indicazione di quanto il modello sia in grado di generalizzare su dati mai visti (meno è addestrato un modello, e più alto sarà il validation error).
- **training error:** è l'errore che ottengo quando testo il modello con il training set, e quindi mi dà un'indicazione di quanto il modello sia in grado di imparare dai dati di addestramento. (più viene addestrato un modello, e più alto sarà il training error)





Queste curve con il passare del tempo, tenderanno ad incontrarsi in un punto.

L'elaborazione dati e l'ottenimento occupa un sacco di tempo e denaro, quindi viene usato per valutare la qualità del modello, prima di cercare di migliorarlo

i principali tipi di comportamento che possiamo osservare sono:

- **Bias Alto:** le due curve si incontrano molto più in alto di quanto vorremmo, quindi abbiamo un underfitting
- **Variance Alto:** le due curve si incontrano dove vorremmo, ma sono molto distanti tra di loro, e quindi abbiamo un overfitting

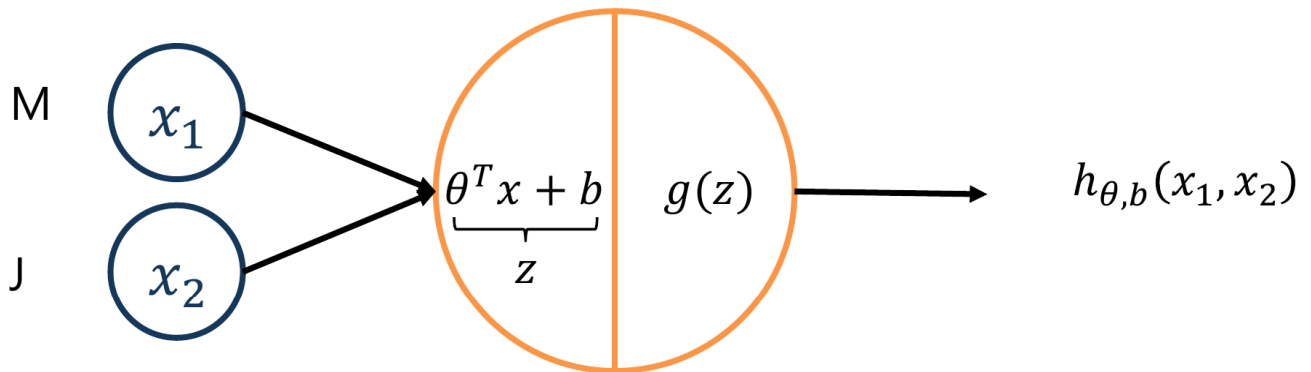


Neural Networks



Modifica della logistic regression

chiamiamo $b = \theta_0$ (*bias*):
$$h_{\theta}(x) = g(\theta^T x + b) = \frac{1}{1 + e^{-(\theta^T x + b)}}$$
 Prendiamo questo concetto



come funziona questo "neurone artificiale"?

con un solo neurone, è di fatto solo una logistic regression (a cui va aggiunto un bias).

Si associa ad ogni variabile un proprio peso, che tramite il training viene modificato, per poter rappresentare meglio i dati.



Multilayer Neural Networks

Possono essere inseriti più strati (**layers**), in modo da poter rappresentare funzioni più complesse, e quindi poter risolvere problemi più complessi.

I modelli più complessi (prendi ChatGPT) hanno [circa 50 transformer layers](#) e quindi sono in grado di rappresentare funzioni molto complesse, e quindi risolvere problemi molto complessi. La complessità del modello aumenta esponenzialmente con il numero di layer, quindi spesso il metodo "forza bruta" non è sempre il più adatto. Si può quindi utilizzare una serie di modelli più semplici, ma estremamente efficienti a riconoscere e risolvere problemi specifici che poi fanno riferimento ad un unico esperto, il cui compito è quello di scegliere il sottomodello più adatto, in modo da poter risolvere problemi complessi, senza dover aumentare troppo la complessità del modello.

A differenza della logistic regression, dove ha solo il gradient descent, nelle reti neurali abbiamo una serie **funzioni di attivazione**, che non sono mai lineari. Quella che noi vedremo più nel dettaglio è la **sigmoid**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

La proprietà interessante della sigmoid è che è una funzione che mappa qualsiasi numero reale in un intervallo tra 0 e 1, e quindi è molto utile per la classificazione, perchè ci consente di interpretare l'output del modello come una probabilità.

L'importanza della non linearità è che ci consente di rappresentare funzioni più complesse: Un modello molto grande, basato funzioni lineari, sarebbe in grado di rappresentare solo funzioni lineari, e quindi non sarebbe in grado di risolvere problemi complessi, mentre un modello basato su funzioni non lineari, come la sigmoid, è in grado di rappresentare funzioni più complesse, e quindi risolvere problemi più complessi.

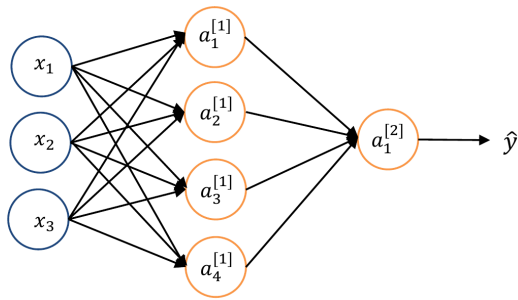
Ma come si decide quanti neuroni e quanti layer usare?

Non è banale come operazione, soprattutto per problemi più complessi.

è sempre buona idea cercare se qualcuno ha già risolto un problema simile, perchè la complessità del modello sarà simile, risparmiando una gran quantità di tempo, e quindi permettendoci di concentrarci sulla rifinitura del modello, invece di doverlo costruire da zero.

Nelle reti neurali *classiche* i neuroni di un layer sono tutti connessi ai neuroni del layer successivo.

Se volessimo calcolare il numero di parametri di questo modello, dovremmo iniziare a trovare il numero di parametri di ogni layer, per poi sommarli tutti assieme aggiungendo anche i bias in ogni livello ad ogni neurone viene associato un peso.



ad esempio, in questa immagine abbiamo 12 pesi nel primo layer (3×4).

Queste informazioni possono essere inserite all'interno di una matrice 4×3 . Se moltiplico una matrice 4×3 (primo layer) per una matrice 3×1 (layer di input), ottengo una matrice 4×1 , che rappresenta l'output del primo layer, che poi viene usato come input per il

secondo layer.

A questo vettore generato, viene poi applicata una funzione di attivazione (sigmoid) che permette di trasformare i valori numerici in un intervallo tra 0 e 1 per tutti i valori all'interno del vettore.

Sucessivamente ci si sposta al secondo layer (nel nostro caso il passaggio finale), e moltiplichiamo i valori di attivazione del primo layer (4×1) per i pesi del secondo layer (1×4). Questa operazione genera un vettore scalare (1×1), che rappresenta l'output del secondo layer, viene poi applicata una funzione di attivazione.

questo ultimo valore è il risultato e quindi output della rete neurale.

Multilayer Neural Networks: Training

I valori dei pesi per alcuni NN specifici sono già noti o ricercati, tuttavia per creare un modello da zero (a meno che non sia un modello molto semplice) è necessario automatizzare il processo di assegnazione dei pesi.

Per ogni passaggio del training, viene fornito alla rete un input, e viene calcolato l'output (**forward step**).

Si confronta questo valore con il valore reale, se è sbagliato (cosa estremamente probabile all'inizio del training, perchè il modello genera output casuali), si calcola l'errore, e si esegue una operazione di **backward step**: grazie ad un algoritmo di ottimizzazione si cerca di cambiare i pesi dei neuroni e bias si cerca di minimizzare l'errore.

Dopo una serie di iterazioni (se il modello è impostato correttamente e i dati sono sufficienti), il modello dovrebbe essere in grado di generare output sempre più vicini al valore reale.

Si ricorda che ogni parametro ha la propria funzione di costo, e quindi ogni parametro viene aggiornato in base al proprio gradiente, in modo da minimizzare la funzione di costo.

è importante notare che se, anche in un singolo layer, un input viene ridotto tra 0 e 1, il modello non sarà mai in grado di rappresentare un output con un valore diverso da quel intervallo